

MS SQL Server Distributed Partitioned Views

By Don Schlichting

This article will explore the use of Distributed Partitioned Views for accessing multiple MS SQL Servers when configured as a Federated Database.

Introduction

When performance gains are needed on very large databases, and your stored procedures are already optimized, and the application is already n-tiered out, and your hardware is already upgraded, its time for distributing the database work over several servers. For SQL Server, this is done by horizontally partitioning large tables over multiple servers. If we think of splitting one table with many columns, into several tables of only a few columns, as Vertical Partitioning, then Horizontal Partitioning is the act of taking one table with many rows, and splitting it into many tables with only a few rows. If these new smaller tables are placed on different servers, it is called a Federated Database configuration. The word Federated is used because all the servers involved may cooperate to balance the processing load. They will act as one federation. Once your data is split among several servers, a new type of statement is needed for retrieving records. These new statements are called Distributed Partitioned Views. They use standard SQL statements, along with the key word UNION, to pull data from all the distributed servers. DML statements (INSERT, UPDATE, and DELETE) can also be used when just a few special rules are observed on the underlying tables. While performance gains are going to vary from application to application, gains of 20% to 30% seem common.

There are three main configuration tasks. Start with attaching all of the servers to each other via Linked Servers, followed by creating matching tables on each server, and finally writing the new views.

In this example, we will distribute the SQL pubs "Authors" table across two different servers. The rules and procedures are identical regardless of the number of servers involved.

Linked Servers

For a detailed explanation of linked servers see previous article [Linked Servers PART1](#).

The first step in creating the federation is to link all of the servers involved together. Start Query Analyzer as "sa," run from the first server. This code will link the second server to the first with an alias of "server2."

```
USE master
GO

EXEC sp_addlinkedserver
    @server = 'server2',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'InfoNet'
```

The @server variable is our alias. The @datasrc is the actual Sql server name. If there was more than one instance on the server, ServerName\InstanceName would have been used.

If the first server will be running all the views, and your Windows login has the appropriate rights to both machines, then special logins are not needed. However, if a different client will be executing the views, then login mappings may be needed. See *Linked Servers PART3* and <http://support.microsoft.com/default.aspx?scid=http://support.microsoft.com:80/support/kb/articles/q238/4/77.asp&NoWebContent=1> for detailed explanations of linked server login options and client problems. We will assume remote clients may be wanted in the future, so a linked server login, mapping the local "sa" account to remote "sa" account, is being used as an example. For production, create a new login rather than expose "sa."

```
EXEC sp_addlinkedsrvlogin 'server2', 'false', 'sa', 'sa', 'secret'
```

To test the link, execute:

```
SELECT *  
FROM server2.pubs.dbo.authors
```

All of the rows from Authors should be returned. We now need to repeat the same from server 2. This will create a link back to server1. Everything is the same except for the server name and alias. Login to the second server's Query Analyzer as "sa" and run:

```
USE master  
GO
```

```
EXEC sp_addlinkedserver  
    @server = 'server1',  
    @srvproduct = 'SQLServer OLEDB Provider',  
    @provider = 'SQLOLEDB',  
    @datasrc = 'dons13'  
GO
```

```
EXEC sp_addlinkedsrvlogin 'server1', 'false', 'sa', 'sa', 'secret'  
GO
```

```
SELECT *  
FROM server1.pubs.dbo.authors
```

If there were additional servers involved, each would require links to every other. The same type of design we had with NT multi-Domain security trusts. If there were four servers in our federation, it would have this landscape:

Server1 has a linked server to Server2, Server3, and Server4.

Server2 has a linked server to Server1, Server3, and Server4.

Server3 has a linked server to Server1, Server2, and Server4.

Server4 has a linked server to Server1, Server2, and Server3.

There is not an automated way to either create, or test these reciprocal links.

By Don Schlichting

Create Table

For this example, we will be working with a subset of the pubs Authors table. We will imagine the table was very large and most of our lookups were done by last name. In this case, we could split the Authors table in half, with one server getting records with a last name beginning with A - M, and the other server getting records N - Z. From server1, create and load a test table.

```
CREATE DATABASE test
GO
```

```
USE test
GO
```

```
CREATE TABLE AuthorsAM(
    au_lname varchar(40) NOT NULL,
    au_fname varchar(20) NULL,
    CONSTRAINT CHK_AuthorsAM CHECK (au_lname < 'N')
)
```

```
GO
```

```
INSERT INTO AuthorsAM
    (au_lname, au_fname)
SELECT au_lname, au_fname
FROM pubs..authors
WHERE au_lname < 'N'
```

From server2, almost identical code is run again. The check will be changed for our new range of last names. Notice the tables do not need to have the same name on each server. Server1s' table is called AuthorsAM, server2s' table is called AuthorsNZ. Our view will take care of sorting this out.

```
CREATE DATABASE test
GO
```

```
USE test
GO
```

```
CREATE TABLE AuthorsNZ(
    au_lname varchar(40) NOT NULL,
    au_fname varchar(20) NULL,
    CONSTRAINT CHK_AuthorsNZ CHECK (au_lname >= 'N')
)
```

```
GO
```

```
INSERT INTO AuthorsNZ
    (au_lname, au_fname)
SELECT au_lname, au_fname
```

```
FROM pubs..authors
WHERE au_lname >= 'N'
```

The critical section of code is the constraint. The check must insure that a row has only one exclusive table in which to be located. When our views are run, the Query Optimizer will use these constraints to determine which servers will receive the distributed work.

Distributed Partitioned Views

The last step is the actual creation of the views. The UNION operator is used to merge the results from both tables into one result set. See BOL "Union operator" for detailed Union explanations and rules.

From server1:

```
CREATE VIEW AllAuthors
AS
SELECT *
FROM AuthorsAM

UNION ALL

SELECT *
FROM server2.test.dbo.AuthorsNZ

GO
```

From server2 the code again is almost identical. Only the server names are changed.

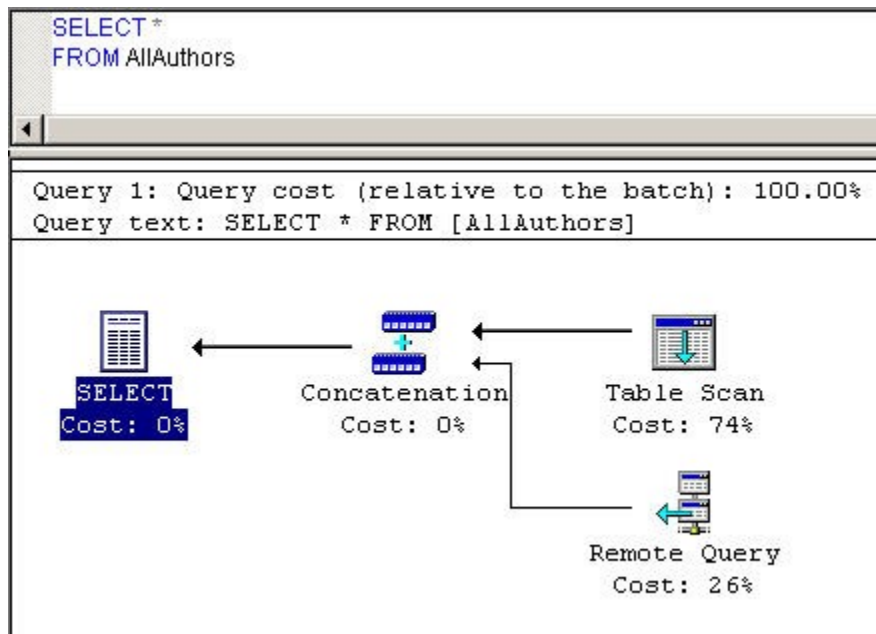
```
CREATE VIEW AllAuthors
AS
SELECT *
FROM AuthorsNZ

UNION ALL

SELECT *
FROM server1.test.dbo.AuthorsAM

GO
```

Running a simple select from server1 produced this Execution Plan:



We can see the local table scan on server1 is merged with a remote query on server1. Our view follows all the standard rules for views. So returning only subsets requires nothing more than a standard where clause:

```
SELECT *  
FROM AllAuthors  
WHERE au_lname BETWEEN 'F' AND 'W'
```

Conclusion

While setting up Federated servers is not a quick task, the performance gains produced by Distributed Partitioned Views on large tables can make it more than worth while. We will continue next month with the requirements for DML on Federated servers, and optimization techniques.